

Spring Security

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        http
            .csrf(csrf->csrf.disable())
            .authorizeHttpRequests(auth ->auth
                .requestMatchers(HttpMethod.GET,"/api/posts/**").permitAll()
                .requestMatchers(HttpMethod.GET,"/api/categories/**").permitAll()
                .requestMatchers(HttpMethod.GET,"/api/tags/**").permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(session->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            );

        return http.build();
    }
}
```

Why @Bean used here?

@Bean means that a `SecurityFilterChain` object will be **created by Spring** and **registered in the application context** as a **singleton** (default scope).

This tells Spring to **use this configuration** instead of its **default security settings**, allowing us to fully **customize how HTTP requests are secured** (e.g., public routes, authentication, JWT, etc.).

ApplicationContext:- It's like a container where Spring keeps all beans. When a request comes in, Spring checks here for how to handle it.

SecurityFilterChain:- It's a set of filters and rules applied to incoming HTTP requests to control access.

How method chaining works here?

Each of these methods (`csrf()`, `authorizeHttpRequests()`, `sessionManagement()`, etc.) returns the **same `HttpSecurity` object**, or something that eventually returns back to it.

SPRING SECURITY + JWT: SESSION MANAGEMENT EXPLAINED

Q1: What happens if I don't use `.sessionManagement()` in my security configuration?

A:

If you don't explicitly configure `.sessionManagement()`, Spring Security will default to: `.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)`

This means Spring will only create an HTTP session **if it thinks it's necessary**, such as for form login or CSRF protection.

Q2: What does `SessionCreationPolicy.IF_REQUIRED` mean?

A:

`IF_REQUIRED` tells Spring:

"Create a session only if required by the current authentication mechanism."

This is the default behavior. If you're using form-based login or need to store authentication info (like `SecurityContext`) across requests, a session will be created.

Q3: When does Spring consider a session as "required"?

A:

Spring may create a session in these cases:

- You're using **form login** or **HTTP Basic**
 - You're using **CSRF protection**
 - You're using `HttpSession` or `@SessionAttributes` in your code
 - You haven't explicitly disabled sessions in a stateless API
-

Q4: If I use JWT authentication and leave session policy as `IF_REQUIRED`, what happens?

A:

Spring may:

- Create a session after the first successful authentication
- Store the `Authentication` object in the session
- **Skip token validation** for future requests because it reuses session

 This **defeats the stateless nature** of JWT and causes inconsistent behavior.

Q5: Why is this a bad practice when using JWT?

A:

JWT is meant to be **stateless**. Each request should:

- Be authenticated **independently**
- Carry its own JWT token
- **Not rely on server-side session storage**

If Spring creates a session, you lose these benefits and introduce security concerns (like session fixation, logout issues, or inconsistent token use).

Q6: What should I do to ensure proper JWT behavior?

✅ **A:**

Always configure Spring Security with:

```
.sessionManagement(session
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

->

This guarantees:

- **No sessions will ever be created**
- Spring **won't store the Authentication** object in a session
- Every request is **validated via JWT**

Q7: What's the best practice for session policy in REST APIs with JWT?

✅ **A:**

http

```
.sessionManagement(session->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

This is **required** for stateless, secure, token-based authentication.

That's a fantastic question — and one that shows you're thinking about **architecture** and **scalability**, not just code. Let's walk through it step by step.

✅ **Question 1: How can Spring Security and JWT work together?**

🔑 **Answer:**

Spring Security is the **framework** that manages security concerns (authentication + authorization), while JWT is a **mechanism for carrying authentication information** in a stateless way.

- Spring Security can use many authentication methods:
 - Session-based
 - HTTP Basic
 - Form login
 - JWT

- OAuth2, etc.

When you plug **JWT** into Spring Security:

- You **authenticate** the user (using Spring Security)
- You **generate a JWT** (custom code)
- You **validate the JWT** in every request (custom filter)
- You **use Spring Security's authorization features** (e.g., `hasRole`, `@PreAuthorize`, etc.)

So Spring Security still controls the **security pipeline**, and JWT just changes **how the authenticated identity is passed across requests**.

✅ **Question 2: If we don't use JWT, what happens?**

Without JWT, Spring Security defaults to **session-based authentication**:

- When a user logs in, Spring creates an **HTTP session**
- It stores the user's **Authentication** (identity) inside the session (**SecurityContext**)
- On every future request, the **session ID** is sent in the cookie
- Spring restores the user's identity from the session

💬 **Problem With This in Modern Apps:**

Problem	Explanation
❌ Not stateless	Every user has a server-side session. More users = more server memory usage.
❌ Not ideal for REST APIs	APIs are supposed to be stateless. Mobile and frontend apps don't always use cookies/sessions.
❌ Doesn't scale well across servers	If your app runs on multiple servers (e.g., in a load-balanced environment), sessions must be shared across servers (via sticky sessions or Redis) — adds complexity.
❌ Harder to integrate with frontend SPAs	Frontend frameworks (React, Angular, etc.) prefer using tokens in headers, not cookies.
❌ Logout is tricky	You'd have to invalidate the session manually or track token invalidation — more effort.

✅ **Question 3: Will my application not scale without JWT?**

📌 **Short Answer:**

It **will still scale**, but **not efficiently**, especially in a microservices or cloud-based environment.

Why?

1. Session-based auth requires server memory

- Each user = memory usage for session
- Harder to run multiple instances of your app

2. You'll need session replication

- To scale horizontally (across servers), you need sticky sessions or store sessions in Redis — which adds complexity.

3. Not ideal for mobile/SPAs

- These clients usually prefer JWT-style APIs where they manage the token and send it in the header.

✅ JWT Is Designed for Scalable, Stateless Systems

- ✅ No server memory needed per user
- ✅ Each request is self-contained (includes token)
- ✅ Works perfectly across multiple instances
- ✅ Ideal for REST APIs, mobile apps, SPAs

1. Why do sessions need to be shared across servers in a load-balanced environment?

✅ The Problem:

When your app runs on **multiple servers (instances)** (e.g., behind a load balancer), each server **has its own memory** and **its own session store**.

HTTP sessions are stored in memory by default.

If a user logs in and their session is stored in **Server A**, but their next request is routed to **Server B**, then:

- Server B has **no idea who the user is**
- The session is **missing**
- The user appears **unauthenticated**

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
    return config.getAuthenticationManager();
}
```

This method **exposes the `AuthenticationManager` as a Spring Bean** in your application context. The `AuthenticationManager` is a **core component in Spring Security** — it is responsible for **authenticating user credentials**, like verifying username and password during login.

Why is it annotated with `@Bean`?

✅ **Answer:**

Because you need to make the `AuthenticationManager` **available to other parts of your app**, like:

- Custom login controller
- JWT authentication filter
- Token generation service

Spring doesn't automatically create this bean for you in Spring Boot 3.x (unlike older versions), so **you have to declare it manually** like this.

`@Bean` means: “Hey Spring, make this method's return value (`AuthenticationManager`) available in the application context so I can inject it anywhere I need it.

```
@Getter
@RequiredArgsConstructor
public class BlogUserDetails implements UserDetails {

    private final User user;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority("ROLE_USER"));
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }
}
```

```

    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}

```

BlogUserDetails

BlogUserDetails is a **custom implementation** of Spring Security's UserDetails interface.

Spring Security needs a standard format to represent a **logged-in user**. That format is defined by the UserDetails interface.

Since you have your own User class (probably a JPA entity with fields like id, email, password, etc.), Spring can't use it directly. So, you wrap it using a custom class that implements UserDetails.

? What is BlogUserDetails?

It's a class that **wraps your User entity** and implements the UserDetails interface so Spring Security can understand your user model.

◆ Its job: Expose your User fields (like email, password, roles) in a way Spring Security understands.

🧠 Line-by-line Explanation

private final User user;

- You store the actual user object from your database here.

- This lets you access things like `user.getId()`, `user.getEmail()`, etc., if needed later.

◆ `getAuthorities()`

```
return List.of(new SimpleGrantedAuthority("ROLE_USER"));
```

Returns a list of roles or permissions the user has.

Here, you're giving **every user** the role `ROLE_USER`.

If you had roles in your `User` entity, you would dynamically convert them here.

```
@RequiredArgsConstructor
public class BlogUserDetailsService implements UserDetailsService {

    private final UserRepository userRepo;

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        User user = userRepo.findByEmail(email)
            .orElseThrow(
                () -> new UsernameNotFoundException("user not found with email:" + email));
        // 🔗 Here is the link: wrapping User in BlogUserDetails
        return new BlogUserDetails(user);
    }
}

-----BlogUserDetailsService.java-----
```

What is **BlogUserDetailsService**?

It's your **custom implementation** of Spring Security's `UserDetailsService`.

◆ Its job: **Load a user from the database using email (or username)** and return a `UserDetails` object

```
Spring Security → BlogUserDetailsService → UserRepository → User
                    ↓
                BlogUserDetails(user) → returned to Spring
```

Why do we do this?

Spring Security is unaware of your `User` class. It only knows how to work with `UserDetails`. So you “wrap” your `User` inside a class (`BlogUserDetails`) that implements `UserDetails`, exposing only the necessary things Spring Security needs, like:

- `getUsername()` → from `user.getEmail()`
- `getPassword()` → from `user.getPassword()`
- `getAuthorities()` → maybe from `user.getRole()` or similar

I don't see a constructor in BlogUserDetails that accepts a User. So how is new BlogUserDetails(user) even working?

```
@RequiredArgsConstructor
public class BlogUserDetails implements UserDetails {
    private final User user;
}
```

@RequiredArgsConstructor – This is a Lombok annotation.

🔧 It **auto-generates a constructor** that accepts all **final** fields as parameters.

✅ How They Are Connected

🔄 Flow:

1. Spring Security needs to authenticate a user.
2. It calls your `BlogUserDetailsService.loadUserByUsername(email)`.
3. Inside that method, you fetch the `User` from the DB.
4. You **wrap** the `User` inside a `BlogUserDetails` object.
5. You return that object to Spring Security.